

ORACLE

Distributed AI Infrastructure Platform

*Turning several physically connected computers into one
AI inference cluster*

Technical Report

Version 0.1 — Engineering prototype

Project Oracle – Distributed AI Infrastructure Platform
Repository <https://github.com/Amouriii/Oracle>
Course AI Academy — Capstone Project
Duration Approximately one week of development
Date August 21, 2026

Team Members

Ahmed Abdelfattah	<i>Project lead — engine, transport, orchestration</i>
-------------------	--

_____	_____
_____	_____
_____	_____

This report describes an academic engineering prototype. Implementation status is stated per component and is derived from the project repository. Oracle is **not** a production system, and no performance numbers are claimed unless they have been measured.

Abstract

Modern large language models routinely require more memory than a single ordinary workstation can provide, which places them out of reach for teams that own several medium-sized machines but no large accelerator. Oracle addresses this gap by treating a small group of physically connected computers as a single AI inference platform. A model is split layer-by-layer across the machines; each machine holds only its own shard of weights and its own slice of the attention key/value cache, and passes intermediate activations to the next machine over a high-speed local link using a compact binary tensor protocol. A master node owns the public interface: it exposes an OpenAI-compatible HTTP API, plans how the model is distributed across the cluster from the model's own metadata and each node's memory budget, tracks node health through UDP heartbeats, and refuses work when the cluster cannot safely serve it. The prototype implements the distributed dataflow, the sharding planner, the health layer and the API surface; concurrent multi-request serving, weight-level model execution and the security layer are specified in this report and are partially implemented or planned. Oracle's expected impact is to make large-model inference reachable on commodity hardware that organisations already own.

1 Introduction

The practical cost of running a large AI model is dominated by a single constraint: the model's parameters, its activations, and its attention cache must all be resident in memory at the same time. A 70-billion-parameter model quantised to roughly four bits per weight occupies on the order of 35–40 GiB of weights alone, before any per-request key/value (KV) cache is allocated. A machine that cannot hold that working set either cannot run the model at all, or falls back to swapping weights from storage on every token, which slows generation by orders of magnitude.

Beyond memory, serving a model imposes three further demands:

- **Compute.** Each generated token requires a full forward pass over every layer. Matrix multiplication throughput, not clock speed, sets the token rate.
- **Storage.** Model files are tens of gigabytes; loading them is a measurable part of start-up time and must be planned, not improvised.
- **Networking.** As soon as a model is split across machines, intermediate activations must cross the network on every token. The link between machines becomes part of the inference path, and its latency is paid once per hop per token.

Relying on a single computer therefore creates a hard ceiling. Buying a larger machine or renting cloud accelerators solves the problem with money rather than with engineering, and neither option is available to every student, laboratory or small organisation. What such groups often *do* have is several ordinary computers.

Oracle is a distributed AI inference platform that combines those machines. Computers are connected over a dedicated high-speed link — in the reference deployment, a Thunderbolt 3 bridge — and cooperate to serve a model that no single member could hold. One machine acts as the *master*: it terminates client requests, decides how the model is laid out across the cluster, monitors the other machines, and returns generated tokens. The others act as *workers*: each owns a contiguous range of model layers and executes only that range. From the outside, the whole cluster looks like one endpoint speaking the OpenAI chat-completions protocol.

This report describes Oracle's design, states precisely which parts of that design are implemented in the current prototype, and sets out what remains.

2 Problem Statement and Objectives

2.1 Problem statement

Given a small number of ordinary computers connected by a fast local link, serve a large AI model that exceeds the memory of any single machine, expose it through a standard API, keep serving while machines come and go, and do so without assuming specialised datacentre hardware.

Three sub-problems follow directly. First, **placement**: the model must be divided so that every machine’s share fits inside its declared memory budget, including the KV cache that grows with sequence length. Second, **coordination**: the machines must exchange activations for every token with low enough overhead that the network does not dominate the token rate. Third, **trust and liveness**: a cluster with one unresponsive machine must detect that fact and fail loudly, rather than blocking indefinitely on a machine that will never answer.

2.2 Objectives

Table 1: Project objectives and how each is addressed in the current prototype.

Objective	Meaning in Oracle	Status
Distributed AI inference	Split one forward pass across several machines	IMPLEMENTED
Large-model support	Layer sharding sized against per-node RAM budgets	IMPLEMENTED
Multi-request serving	Serve several client requests at once	PARTIALLY IMPLEMENTED
Parallel execution	Overlap work instead of a strict serial pipeline	PLANNED
Intelligent orchestration	Place layers from model metadata + node resources	PARTIALLY IMPLEMENTED
Model recognition	Read model architecture and size from the model file	IMPLEMENTED
Heartbeat / monitoring	Detect unresponsive nodes and act on it	IMPLEMENTED
Security	Authenticate clients and nodes, bound resource usage	PLANNED
Deployment	Reproducible build and native cluster binaries	IMPLEMENTED
OpenAI-compatible API	Existing clients work unchanged	IMPLEMENTED

Status labels are used consistently throughout this report: **IMPLEMENTED** means the behaviour exists in the repository and is exercised by tests or scripts; **PARTIALLY IMPLEMENTED** means the mechanism exists but is incomplete or placeholder in an important respect; **PLANNED** means it is designed and specified here but not yet coded; **FUTURE** means it is a longer-term direction.

3 Oracle System Architecture

Oracle is a master/worker system. Exactly one node carries the **master** role and owns everything client-facing and cluster-wide; every other node is a worker that executes a slice of the model and nothing else. All nodes run the same engine library and differ only in configuration and entry-point binary.

Master. Loads the cluster description, asks the shard manager for a placement, builds the execution graph, opens the tensor transport and the heartbeat socket, and then serves HTTP. It performs embedding and the first layer range itself, so the master is a compute participant, not only a coordinator.

Workers. Each worker connects to its predecessor in the chain, blocks on the transport, runs its own layer range over each activation tensor it receives, and forwards the result to its successor. The final worker additionally applies the output projection (`lm_head`) and sends logits back to the master.

Transport. A single packed 76-byte header (`TensorHeader`, magic `ORCL`) followed by a raw payload. There is no JSON or protobuf on the hot path; header and body are written together

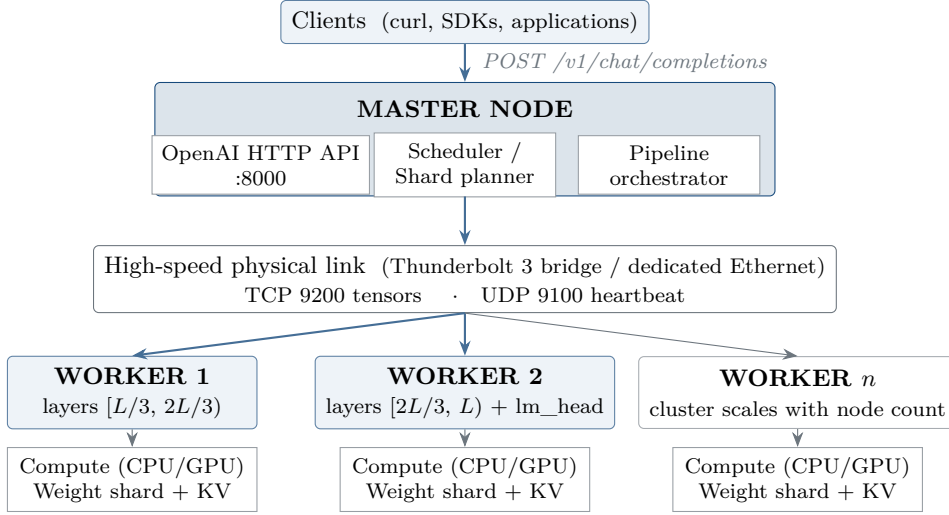


Figure 1: Oracle cluster architecture. The master owns the API, placement and orchestration; workers own layer shards and local KV state. Activations travel master $\rightarrow$ worker $\rightarrow \dots \rightarrow$ master once per generated token.

with `writetv` and the payload carries a CRC32. For two processes on the same machine, a lock-free single-producer/single-consumer shared-memory ring replaces the socket.

4 Distributed AI Inference

4.1 How a model is split

Oracle divides a model along its *layer* axis. Given a model with L transformer blocks and n nodes, the shard manager assigns each node a contiguous range so that the ranges tile $[0, L)$ exactly. For a 70B-class model with $L = 80$ and three nodes, each node owns roughly 26–27 consecutive layers and about one third of the weights.

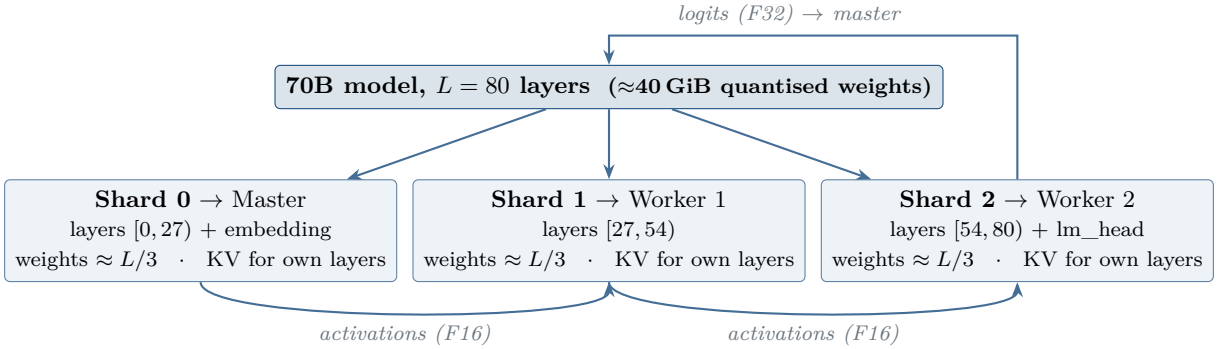


Figure 2: Layer-wise model sharding. Each node stores only its own weight shard and the KV cache for its own layers; only a single hidden-state vector crosses the link per hop per token.

Distributed memory. Two distinct memory pools are planned per node: the weight shard (proportional to the node’s layer count) and the KV cache. The KV cache is sized from the model’s own geometry — for each local layer, $2 \times n_{kv} \times d_{head} \times \text{max_seq}$ elements for keys and values — and stays permanently on the node that owns those layers. It is never transferred, which is precisely why per-token network traffic stays small.

Activation transfer. What crosses the link is one hidden-state vector per hop per token. For a hidden dimension of 8192 in half precision, that is about 16 KiB. On a Thunderbolt-3-class

Table 2: System components and implementation status.

Component	Module	Responsibility	Status
Tensor transport	<code>tb3-transport</code>	Packed binary tensor frames over TCP, shared-memory ring for the local hop, UDP heartbeat socket	IMPLEMENTED
Shard manager	<code>mem-shard-manager</code>	Layer assignment, KV cache planning, memory-budget feasibility, host memory-pressure sampling	IMPLEMENTED
Orchestrator	<code>orchestrator-core</code>	Pipeline DAG, token loop, heartbeat bookkeeping, OpenAI HTTP server	IMPLEMENTED
Node runners	<code>metal-cpu-runner</code>	Pluggable compute back ends (Accelerate BLAS, Metal GEMM, llama.cpp/GGUF)	PARTIALLY IMPLEMENTED
Master binary	<code>oracle-engine-master</code>	Entry point: config, API, cluster bring-up	IMPLEMENTED
Worker binary	<code>oracle-engine-worker</code>	Entry point: receive, compute, forward	IMPLEMENTED
Network bench	<code>oracle-engine-bench-net</code>	Link throughput and round-trip measurement tool	IMPLEMENTED
Dashboard UI	—	Cluster health and utilisation visualisation	PLANNED
Security layer	—	Authentication, rate limiting, node identity	PLANNED

link this is a small payload, and the cost per hop is dominated by round-trip latency rather than bandwidth.

Worker computation and result propagation. A worker receives the tensor, validates the magic number and CRC, runs its layer range while updating its local KV cache, and forwards the result. The last node in the chain produces the vocabulary-sized logits vector and sends it to the master, which samples the next token, embeds it, and starts the next round. Generation stops at the end-of-text token or when `max_tokens` is reached.

4.2 Which kinds of parallelism are actually used

The word “parallelism” covers several different techniques, and it is important not to conflate them.

4.3 Honest note on model execution

The distributed dataflow — sharding, transport, pipeline traversal, KV planning, sampling and streaming — is implemented and testable end to end. The *numerics* inside a layer are not yet those of a real transformer: the Accelerate back end currently loads identity matrices as placeholder layer weights, and the llama.cpp back end reads real GGUF metadata but delegates the forward pass to the same placeholder path unless the library is linked at build time. In other words, Oracle today moves the right tensors, of the right shapes, along the right path, at the right time, but does not yet produce meaningful text. Wiring real GGUF weight execution into the runner interface is the single most important next step and is listed as such in Section 13.

Table 3: Forms of parallelism: definition and status in Oracle.

Form	What it means	Status
Pipeline parallelism	Consecutive layer ranges live on different machines; a token’s forward pass visits them in sequence. This is Oracle’s core mechanism.	IMPLEMENTED
Model (tensor) parallelism	A <i>single</i> layer’s matrices are split across machines, which then perform a collective reduction. Oracle does not do this; a layer always executes entirely on one node.	FUTURE
Request-level parallelism	Several independent client requests progress at the same time. The HTTP server is multi-threaded and the wire protocol carries a per-request sequence identifier, but the orchestrator currently shares one KV cache and one pipeline ring, so requests are not yet isolated from each other.	PARTIALLY IMPLEMENTED
Pipeline overlap	While node 2 works on token t , node 1 already starts token $t+1$, so nodes are not idle. Not yet implemented: the current loop is strictly sequential per token.	PLANNED

5 Handling Large Models and Many Requests

Oracle scales along two independent axes, and the two must not be confused: one is about *how big* a model can be, the other about *how many* clients can be served.

5.1 Dimension A — model scalability

Adding a node adds its RAM budget to the cluster’s pooled capacity. With n nodes each declaring a budget B_i , the shard manager accepts a placement only if, for every node, the weights of its assigned layers plus that node’s KV cache fit within B_i . This check is performed before any weights are loaded, so an infeasible cluster is rejected at planning time with a specific reason rather than failing later with an out-of-memory error. A companion pressure monitor samples real host memory and refuses allocations that would overcommit the machine even when the configured budget would allow them.

The consequence is straightforward: a model that does not fit on one machine may fit on three, and Oracle can tell you *before* you try.

5.2 Dimension B — request scalability

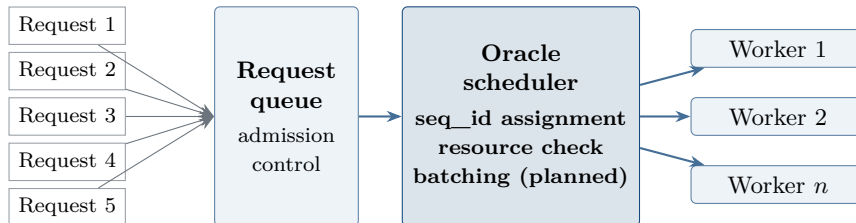


Figure 3: Multi-request scheduling model. Requests are admitted, given a sequence identifier, and dispatched onto the shared worker pipeline. The queue and scheduler stages are the target design; the current build exposes a thread-pooled HTTP front end over a single shared pipeline.

The intended behaviour, and its present state, are as follows.

- **Request queue.** Incoming requests are accepted into a bounded queue rather than each spawning unbounded work. **PLANNED**
- **Scheduling.** Each request receives a unique sequence identifier that travels inside every tensor header, so any node can attribute an activation to the request it belongs to. The identifier and its wire field exist today. **PARTIALLY IMPLEMENTED**
- **Concurrency.** The HTTP layer already dispatches requests onto a thread pool. What is missing is per-request isolation inside the engine: the KV cache and the pipeline ring are currently shared, so concurrent requests would interfere. Giving each sequence its own KV slice and its own in-flight slot on the ring is the concrete work item. **PARTIALLY IMPLEMENTED**
- **Batching.** Grouping the decode steps of several sequences into one matrix multiplication is the standard way to raise throughput, and the transport already tolerates variable payload sizes. Not implemented. **PLANNED**
- **Resource allocation.** KV memory per sequence is computed from model geometry, so the maximum number of concurrent sequences a node can hold is a derivable quantity rather than a guess. **IMPLEMENTED** (planner)
- **Avoiding unnecessary serialisation.** A pipeline whose stages run strictly one after another leaves $n-1$ of n nodes idle at any instant. Overlapping consecutive tokens or sequences across stages is what turns the chain into a true pipeline. **PLANNED**

No claim is made about how many simultaneous requests Oracle can serve. That number depends on model geometry, sequence length and node memory, and it has not been measured; see Section 12.

6 AI Orchestrator

The orchestrator is the part of the master that turns a description of the cluster and a description of the model into an executable plan, and then keeps that plan honest at run time.

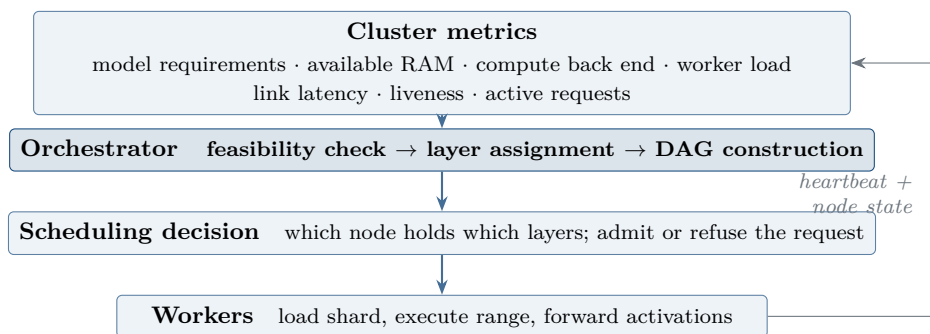


Figure 4: Orchestration loop. Cluster and model facts produce a placement; worker state feeds back and can invalidate it.

Inputs the orchestrator uses today. Model geometry (layer count, hidden dimension, attention head counts, vocabulary, context length, bytes per layer), each node’s declared RAM and VRAM budget, the node’s role, and live heartbeat state. From these it produces a contiguous layer assignment, a per-node KV layout, and a linear execution DAG in which the first node performs embedding and the last performs the output projection.

Inputs the design calls for but does not yet consume. Measured worker load, measured link latency, and the count of in-flight requests per node. These are listed in Figure 4 because they belong in the decision, and because the transport already has the hooks to report them — but the current placement policy does not read them.

Policy. Placement today is deterministic and resource-based: split the layer range as evenly as possible across the configured nodes, then verify each node against its budget and refuse the plan

if any node does not fit. This is predictable and easy to reason about, which is the right trade-off for a one-week prototype. Weighting the split by each node’s actual capability (a machine with twice the RAM taking twice the layers), and later learning a placement policy from observed latencies, are natural extensions (Section 13) — not present behaviour.

At run time the orchestrator enforces one hard rule: before starting generation it re-evaluates heartbeat state, and if any configured worker is marked dead the request is rejected immediately with a `worker_dead` error. The cluster never quietly waits on a machine that is gone.

7 Model Recognition

Oracle does not require the operator to describe a model by hand. Given a model file in GGUF format — the container used by the llama.cpp ecosystem — the engine parses the file’s key/value metadata block and extracts the architecture facts it needs.

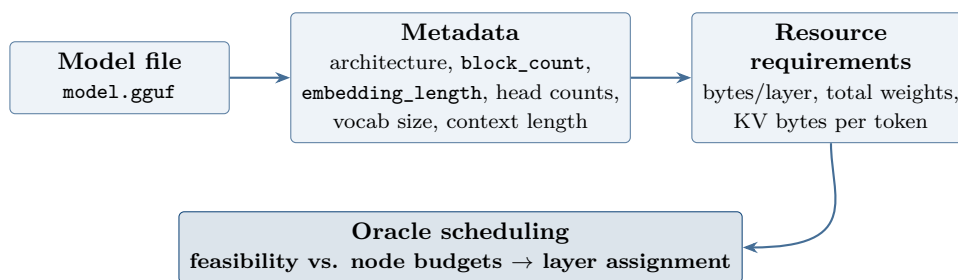


Figure 5: From model file to placement decision. Model metadata is read directly from the file and converted into concrete per-node memory requirements.

- **Architecture and layers.** The architecture name and block count are read from the file header; the block count directly determines how the layer range is divided. **IMPLEMENTED**
- **Parameter count and quantisation.** Where the file declares a parameter count it is used; otherwise the total is derived from the file size on disk, which already reflects the quantisation format. Bytes per layer is then total weight bytes divided by layer count. **IMPLEMENTED**
- **Memory requirements.** Weight bytes per node follow from bytes per layer; KV bytes follow from head counts, head dimension, context length and activation data type. Both feed the feasibility check. **IMPLEMENTED**
- **Runtime compatibility.** The runner interface abstracts three back ends (Accelerate BLAS, Metal, llama.cpp) selected by a command-line flag. Automatic selection of the best back end for a given model and host is not implemented. **PLANNED**

If no model file is supplied, the geometry falls back to the values in the cluster configuration file, which lets the placement logic and the transport be exercised without downloading tens of gigabytes of weights — a deliberate convenience during development.

8 Networking and Heartbeat

8.1 Physical connectivity

The reference deployment connects machines over a **Thunderbolt 3 bridge** rather than over the office network. The reasons are latency and isolation: a direct cable between two machines removes switch hops and shared contention from the inference path, and the cluster’s tensor traffic never touches the general network. The design does not depend on Thunderbolt specifically — any dedicated low-latency link, such as a private 10 GbE segment, serves the same role.

Table 4: Cluster network endpoints in the reference three-node configuration.

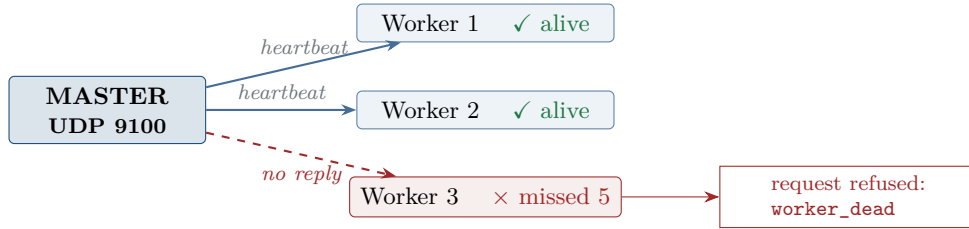
Role	Address	Ports	Traffic
Master	10.10.0.1	TCP 8000 / TCP 9200 / UDP 9100	Client API / tensors / heartbeat
Worker 1	10.10.0.2	TCP 9200 / UDP 9100	Tensors / heartbeat
Worker 2	10.10.0.3	TCP 9200 / UDP 9100	Tensors / heartbeat

8.2 Data transfer and overhead

Tensors are sent as a packed 76-byte header followed by the raw payload, written in a single vectored write so that header and body are not copied together first. `TCP_NODELAY` is enabled, because a 16 KiB activation must be sent immediately rather than waiting for Nagle coalescing. Where two Oracle processes share a machine, the socket path is replaced by a lock-free shared-memory ring, removing the kernel round trip entirely.

Communication overhead per generated token is one activation hop per pipeline stage plus one logits return to the master. This is small in bytes but is paid on *every* token, so the meaningful metric is round-trip latency, not bandwidth. The repository ships a dedicated bench tool that measures exactly this ping-pong; the project’s own target gates are > 8 Gbps of TCP throughput on the bridge and a decode round trip well under 5 ms. These are design targets, not results — see Section 12.

8.3 Heartbeat and failure detection

**Figure 6:** Heartbeat-based liveness. Each node broadcasts a small datagram every 200 ms; five consecutive missed intervals mark a node dead, and the next request is refused rather than blocked.

Every node sends a compact UDP datagram carrying the protocol magic number and its node identifier to each peer at a fixed interval (200 ms by default). Receipt resets that node’s miss counter and refreshes its last-seen timestamp. A periodic sweep increments the miss counter of any node whose last-seen timestamp is older than `interval × misses`, and marks the node dead once the threshold (5 by default) is crossed.

The value of this is concrete. Without it, a request routed through a crashed worker blocks on a socket read until a multi-second timeout expires, and the client sees a hang. With it, the orchestrator checks liveness *before* committing to a generation and returns a clear error immediately. The current prototype detects failure and refuses work; it does **not** re-shard the model onto the surviving nodes or retry elsewhere. Automatic recovery is future work (Section 13).

9 Security

Oracle’s threat model is modest and stated plainly: the cluster’s internal link is assumed to be physically private, while the master’s HTTP port may be exposed to a wider network. The controls below are grouped by what actually exists.

Two limitations deserve emphasis rather than burial. First, the current HTTP server performs

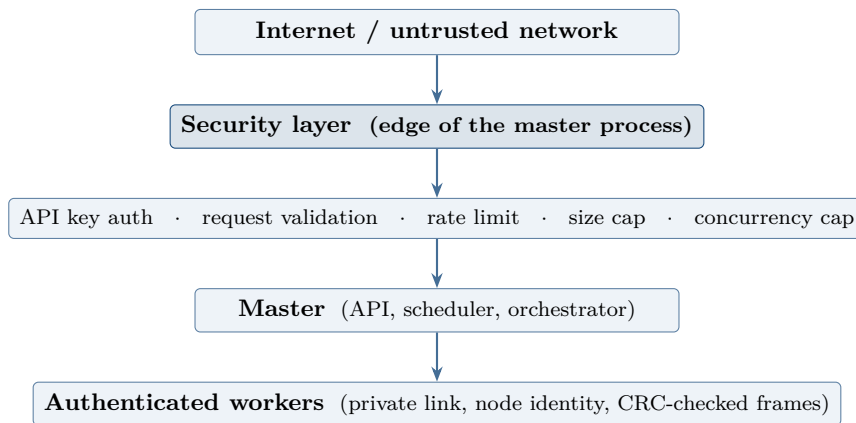


Figure 7: Security architecture. Client-facing controls terminate at the master; the worker mesh is a separate trust domain reached only over the private link.

no authentication: anyone who can reach port 8000 can issue a request, so the prototype must be run on a trusted network or behind a reverse proxy that supplies these controls. Second, heartbeat datagrams are **unauthenticated**, so a host on the internal link could forge liveness for a node that is actually down. Both are direct consequences of the one-week scope, and both are addressed by the planned items above. Oracle does not offer enterprise-grade protection and should not be described as if it did.

10 API and User Interface

10.1 OpenAI-compatible API

The master exposes an HTTP interface that mirrors the OpenAI chat-completions protocol on port 8000.

```

GET /health -> {"status":"ok"}
GET /v1/models -> list of models served by this cluster
POST /v1/chat/completions -> chat completion (streaming or buffered)
POST /v1/completions -> plain completion (same handler)
  
```

```

curl -sS http://10.10.0.1:8000/v1/chat/completions \
  -H 'Content-Type: application/json' \
  -d '{"model":"Oracle",
    "messages":[{"role":"user",
                  "content":"Explain pipeline parallelism."}],
    "max_tokens":128,
    "stream":true}'
  
```

When **stream** is true the server replies with server-sent events, emitting one **chat.completion.chunk** per token as it is produced and terminating with **data: [DONE]**, exactly as clients expect.

Compatibility is a deliberate design decision rather than a convenience. Because the wire format is the one the ecosystem already speaks, existing chat front ends, SDKs, agent frameworks and simple **curl** scripts point at Oracle by changing a base URL and nothing else. The distributed cluster behind the endpoint is invisible to the client, which is the entire point: complexity stays inside the platform. The parsing in the current build is a lightweight hand-rolled field extractor rather than a full JSON parser, and usage accounting is a placeholder — both are noted as hardening work.

Table 5: Security mechanisms and their implementation status.

Mechanism	Behaviour	Status
Frame integrity	Every tensor frame carries protocol magic, version and a CRC32 over the payload; malformed or corrupted frames are rejected	IMPLEMENTED
Resource admission	Allocation is refused when it would exceed a node's declared budget or overcommit host memory, which bounds memory-exhaustion damage	IMPLEMENTED
Liveness gating	Requests are refused when a required node is unreachable, preventing work from being routed into a hole	IMPLEMENTED
Network isolation	Tensor and heartbeat traffic ride a dedicated private link, not the general network	IMPLEMENTED (deployment)
API authentication	Bearer-token/API-key check on every /v1/* request	PLANNED
Request validation	Strict schema validation and bounded field parsing on the JSON request body	PLANNED
Rate limiting	Per-client request rate ceiling	PLANNED
Request-size limits	Maximum body size and maximum <code>max_tokens</code>	PLANNED
Concurrency limits	Cap on simultaneous in-flight generations per client and cluster-wide	PLANNED
DDoS mitigation	Connection caps, timeouts and early rejection of oversized or unauthenticated traffic	PLANNED
Worker authentication	Shared-secret or certificate-based node identity so that only known workers join the mesh	PLANNED
Secure node communication	Encrypted or authenticated tensor channel (currently plaintext on a private link)	FUTURE
Model integrity validation	Checksum verification of model files before load	PLANNED
Security event logging	Structured audit log of authentication failures, rejections and node state changes	PLANNED

10.2 Conceptual dashboard

The following mock-up shows the operator view Oracle is designed to expose. It is a **PLANNED** component: no user interface exists in the repository today, and all values shown are placeholders.

ORACLE AI PLATFORM	
Model:	Qwen / Llama / ... (80 layers, Q4)
Cluster:	
Master	●-- Worker 1 ●-- Worker 2 ●
Active requests:	XX
Queued requests:	XX
CPU:	XX%
GPU:	XX%
RAM:	XX%
Network:	XX Gbps
Security:	✓
Heartbeat:	✓

The interface is intended to visualise cluster health (which nodes are alive and how the layers are distributed), the loaded model and its geometry, request pressure (active versus queued), per-node resource utilisation, link throughput, and the state of the security controls — that is, the same facts the orchestrator already tracks internally, surfaced for a human operator.

11 Deployment

Oracle is deployed as two cooperating processes – one master, one worker per additional machine – driven by a single shared cluster description.

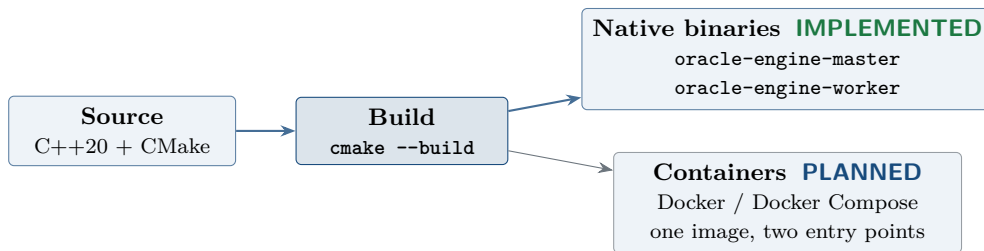


Figure 8: Deployment paths. Native binaries are the supported route today; containerisation is designed but not present in the repository.

Native. The project builds with CMake into three executables: `oracle-engine-master` (config, API, cluster bring-up), `oracle-engine-worker` (layer execution) and `oracle-engine-bench-net` (link measurement), plus an install target that places them on `PATH`. All nodes share one `cluster.toml` that declares the model, the node list, their addresses and their memory budgets; each process is told which node it is with `-id`. A single-machine smoke mode (`-single`) collapses the cluster to one node for development.

```
cmake -S . -B build -DORACLE_BUILD_PYTHON=OFF
cmake --build build -j
./build/oracle-engine-master --config configs/cluster.toml --id 0
./build/oracle-engine-worker --config configs/cluster.toml --id 1
```

An optional Python extension module exposes the engine for scripting, and a small Python helper prints the cluster table from the same configuration file.

Containerised. A container image per role, orchestrated with Docker Compose and sharing a host network so the dedicated link remains visible, is the intended packaging for reproducible deployment. No Dockerfile exists in the repository yet; this is **PLANNED**.

Platform note. The current build targets macOS: the transport uses `kqueue`, the CPU back end uses Apple’s Accelerate framework, and the GPU back end uses Metal. Porting the event loop to `epoll` and the BLAS calls to a portable library is required for Linux support.

12 Testing and Evaluation

12.1 What is tested today

The repository ships five automated unit tests, run through CTest, covering the shared-memory ring buffer, the wire protocol encode/decode round trip, the shard plan (layer coverage, KV sizing, feasibility against budgets), a TCP transport round trip between two sockets, and a tiny end-to-end pipeline that builds a DAG, runs a GEMM through both the CPU and GPU back ends, and generates tokens. Four staged validation scripts cover network benchmarking, allocation dry-run, tensor pass-through and an end-to-end HTTP smoke test.

12.2 Evaluation plan

The metrics below are the ones that matter for a distributed inference platform. No values are reported, because none have been measured on the target cluster. Where the project has defined an internal target, it is shown as a gate, not as a result.

Table 6: Evaluation metrics, method and current values.

Metric	Method	Target gate	Value
Network bandwidth	iperf3 + oracle-engine-bench-net over the bridge	> 8 Gbps	<i>To be measured.</i>
Network latency	Activation-sized ping-pong round trip (16 KiB F16)	$\ll$ 5 ms	<i>To be measured.</i>
Model loading time	Wall clock from process start to first ready state	—	<i>To be measured.</i>
Inference latency	End-to-end wall clock per request	—	<i>To be measured.</i>
Time to first token	Client timestamp of the first SSE chunk	—	<i>To be measured.</i>
Throughput (tokens/s)	Tokens generated / elapsed decode time	—	<i>To be measured.</i>
Concurrent requests	Maximum sustained in-flight requests without error	—	<i>To be measured.</i>
CPU / RAM / GPU utilisation	Per-node sampling during a sustained run	—	<i>To be measured.</i>
Worker failure detection	Kill a worker; time until it is marked dead and a request is refused	$\leq$ interval $\times$ misses	<i>To be measured.</i>
Request success rate	Successful responses / total requests under load	—	<i>To be measured.</i>

Two of these deserve a note on interpretation. *Time to first token* is dominated by prefill and by the number of pipeline hops, so it is the metric most sensitive to adding nodes — adding a machine buys memory but costs latency. *Worker failure detection* is directly predictable from configuration (heartbeat interval $\times$ miss threshold, i.e. one second at the defaults) and is therefore the easiest end-to-end behaviour to verify experimentally.

13 Limitations, Future Work and Conclusion

13.1 Limitations

- **One-week scope.** The project was built in approximately one week. Breadth of the distributed design was prioritised over depth in any single component.
- **Model execution is not complete.** The distributed dataflow is real; the layer numerics are placeholder. Oracle does not yet produce meaningful generated text (Section 5.3).
- **Hardware.** Development targeted a small number of macOS machines with modest discrete GPUs. Weights and KV live in system RAM; the GPUs act as a small cache, not as the primary compute device.
- **Cluster size.** The design and the configuration have been exercised at three nodes. Behaviour at larger scale is untested, and a strictly linear pipeline is unlikely to be the right topology beyond a handful of nodes.
- **Model coverage.** GGUF metadata parsing targets llama-family models. Other architectures and container formats are unhandled.
- **Concurrency.** Per-request isolation inside the engine is incomplete, so genuine multi-request serving is not yet demonstrated.
- **Production hardening.** No authentication, no TLS, no rate limiting, no automatic recovery, no Linux support, and no measured performance figures.

13.2 Future work

1. **Real weight execution.** Wire GGUF weight loading and transformer kernels into the runner interface so that shards execute genuine layers. This unlocks every other measurement in Table 6.

2. **Per-request isolation and continuous batching.** Give each sequence its own KV slice and pipeline slot, then batch decode steps across sequences to raise throughput.
3. **Improved KV-cache management.** Paged, block-allocated KV storage so that memory is proportional to the tokens actually generated rather than to the maximum context length.
4. **Automatic model placement.** Weight the layer split by each node’s measured capability and observed link latency instead of splitting evenly.
5. **Smarter scheduling.** Feed live worker load and queue depth into placement and admission decisions; a learned policy is a longer-term possibility.
6. **Automatic failure recovery.** Re-shard onto surviving nodes and replay the affected request instead of refusing it.
7. **Stronger security.** Implement the planned rows of Table 5, starting with API authentication, request limits and worker identity.
8. **Broader hardware and model support.** Linux (epoll, portable BLAS, CUDA), and more model architectures.
9. **Easier deployment.** Container images, Compose manifests and a one-command cluster bring-up, plus the operator dashboard.

13.3 Conclusion

Oracle demonstrates the concept of turning multiple physically connected computers into a co-ordinated AI inference platform capable of distributing large AI workloads while supporting concurrent requests, orchestration, monitoring, and security. Within a one-week scope the project delivers the parts that are genuinely hard to retrofit: a compact binary tensor protocol with integrity checking, a memory-aware sharding planner that refuses infeasible clusters before they fail, a pipeline orchestrator that traverses a multi-machine layer graph, heartbeat-based liveness that turns a hang into a clear error, and a standard OpenAI-compatible endpoint that hides all of it from the client.

What remains is equally clear, and this report has been explicit about it: executing real model weights, isolating concurrent requests, and implementing the security layer. Those are additions to a working skeleton rather than a change of direction — which is the most useful thing a one-week prototype can establish.

References

- [1] A. Vaswani et al., “Attention Is All You Need,” *NeurIPS*, 2017.
- [2] Y. Huang et al., “GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism,” *NeurIPS*, 2019.
- [3] M. Shoeybi et al., “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” arXiv:1909.08053, 2019.
- [4] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” *SOSP*, 2023.
- [5] G.-I. Yu et al., “Orca: A Distributed Serving System for Transformer-Based Generative Models,” *OSDI*, 2022.
- [6] A. Borzunov et al., “Petals: Collaborative Inference and Fine-tuning of Large Models,” arXiv:2209.01188, 2022.
- [7] G. Gerganov et al., “llama.cpp and the GGUF model format,” <https://github.com/ggml-org/llama.cpp>.
- [8] Y. Hirose, “cpp-http-lib: a C++11 single-file header-only HTTP library,” <https://github.com/yhirose/cpp-http-lib>.
- [9] OpenAI, “API Reference: Chat Completions,” <https://platform.openai.com/docs/api-reference/chat>.